// Security Assessment

10.29.2025 - 10.30.2025

Earn v2 - Configurable Deposit Cap per User

Blueprint Finance

HALBORN

Earn v2 - Configurable Deposit Cap per User - Blueprint Finance

Prepared by: HALBORN

Last Updated 10/31/2025

Date of Engagement: October 29th, 2025 - October 30th, 2025

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
5	0	0	1	2	2

TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Risk methodology
5. Scope
6. Assessment summary & findings overview
7. Findings & Tech Details
   - 7.1 Incorrect use of converttoassets() instead of previewmint() may underestimate asset requirements
   - 7.2 Floating and outdated pragma
   - 7.3 Inconsistent behavior between canuserdeposit() and _checkdepositlimit() causes unexpected transaction reverts
   - 7.4 Use of ownable instead of ownable2step may lead to ownership transfer risks
   - 7.5 Unused sender and totalassets parameter in premint() function

1. Introduction

`Blueprint Finance` engaged `Halborn` to conduct a security assessment on their smart contracts beginning on October 29, 2025 and ending on October 30, 2025. The scope of this assessment was limited to the smart contracts provided to the Halborn team. Commit hashes and additional details are documented in the Scope section of this report.

`Blueprint Finance's` **UserDepositCapHook** contract is a modular access-control hook designed to enforce per-user deposit caps across multiple ERC-4626 vaults. Built with OpenZeppelin's security standards, it integrates seamlessly with vaults to validate deposits and mints before execution, ensuring no user exceeds the configured asset cap for a specific vault. The contract provides flexible per-vault configuration, owner-managed deposit limits, and automated validation through standardized pre-hook checks.

2. Assessment Summary

`Halborn` was provided 2 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the smart contracts.
- Ensure that smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce both the likelihood and impact of potential risks, which were mostly addressed by the `Blueprint` team. The primary suggestions included:

- `Replace convertToAssets() with previewMint() to ensure the calculation rounds up.`
- `Keep the compiler versions consistent in all the smart contract files.`
- `Ensure consistent logic across canUserDeposit() and _checkDepositLimit() when the deposit cap is zero.`

3. Test Approach And Methodology

`Halborn` performed a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is essential to uncover flaws in logic, process, and implementation, automated testing techniques enhance coverage of smart contracts and can quickly identify issues that do not follow security best practices.

The following phases and associated tools were used throughout the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough of the smart contracts to identify potential logic issues.
- Manual testing of all core functions, including **createCampaign**, claim to validate expected behavior and identify edge-case vulnerabilities.
- Local testing to simulate contract interactions and validate functional and security assumptions.
- Local deployment and testing with Foundry.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: earn-v2-core

(b) Assessed Commit ID: 04b28ca

(c) Items in scope:

- src/periphery/hooks/UserDepositCapHook.sol
- src/periphery/interface/IUserDepositCapHook.sol
- src/interface/IHook.sol

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- 98eb69b
- 019daaf

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
0	0	1	2

INFORMATIONAL
2

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
INCORRECT USE OF CONVERTTOASSETS() INSTEAD OF PREVIEWMINT() MAY UNDERESTIMATE ASSET REQUIREMENTS	MEDIUM	SOLVED - 10/29/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
FLOATING AND OUTDATED PRAGMA	LOW	RISK ACCEPTED - 10/30/2025
INCONSISTENT BEHAVIOR BETWEEN CANUSERDEPOSIT() AND _CHECKDEPOSITLIMIT() CAUSES UNEXPECTED TRANSACTION REVERTS	LOW	SOLVED - 10/29/2025
USE OF OWNABLE INSTEAD OF OWNABLE2STEP MAY LEAD TO OWNERSHIP TRANSFER RISKS	INFORMATIONAL	ACKNOWLEDGED - 10/30/2025
UNUSED SENDER AND TOTALASSETS PARAMETER IN PREMINT() FUNCTION	INFORMATIONAL	ACKNOWLEDGED - 10/30/2025

7. FINDINGS & TECH DETAILS

7.1 INCORRECT USE OF CONVERTTOASSETS() INSTEAD OF PREVIEWMINT() MAY UNDERESTIMATE ASSET REQUIREMENTS

// MEDIUM

Description

In the `preMint()` function, the contract currently uses `convertToAssets()` to determine the equivalent asset amount for a given number of shares. However, per the ERC-4626 standard:

- `convertToAssets()` **rounds down**, giving the *best-case* (minimum) asset requirement.
- `previewMint()` **rounds up**, giving the *worst-case* (maximum) asset requirement that the user will actually pay to mint the specified shares.

Since this hook is meant to **validate whether a user can mint** without exceeding their deposit cap, using `convertToAssets()` could cause **underestimation of asset input**, letting users slightly bypass their cap due to rounding differences.

```
123 function preMint(address sender, uint256 shares, address receiver, uint256 totalAssets) extern
124     uint256 assets = IERC4626(msg.sender).convertToAssets(shares);
125     _checkDepositLimit(IERC4626(msg.sender), receiver, assets);
126 }
```

In short, if `convertToAssets()` rounds down, a user might mint shares that push their actual asset value above the intended cap.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

Replace `convertToAssets()` with `previewMint()` to ensure the calculation rounds **up**, preventing deposit cap bypass via rounding discrepancies.

Remediation Comment

SOLVED: The **Blueprint team** resolved the issue by replacing `convertToAssets` with `previewMint`.

Remediation Hash

https://github.com/Blueprint-Finance/earn-v2-core/commit/98eb69bb828b289c932781fc6a03a465c6688a6e

7.2 FLOATING AND OUTDATED PRAGMA

// LOW

Description

Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities. The contract allowed floating or unlocked pragma to be used, i.e., >= 0.8.24. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.29 pragma version.

Remediation Comment

RISK ACCEPTED: The **Blueprint team** made a business decision to accept the risk of this finding and not alter the contracts.

7.3 INCONSISTENT BEHAVIOR BETWEEN CANUSERDEPOSIT() AND _CHECKDEPOSITLIMIT() CAUSES UNEXPECTED TRANSACTION REVERTS

// LOW

Description

The current implementation introduces inconsistent logic between `canUserDeposit()` and `_checkDepositLimit()` when the vault's user deposit cap is set to zero. Specifically, `canUserDeposit()` returns `true` when `userDepositCapByVault[_vault] == 0`, implying that deposits are allowed. However, during an actual deposit, the `preDeposit()` and `preMint()` hooks invoke `_checkDepositLimit()`, which **reverts** with `VaultNotConfigured(address(_vault))` when the same condition occurs.

The `canUserDeposit()` treats cap==0 as "no cap" and returns true, but `_checkDepositLimit()` treats cap==0 as "not configured" and reverts — causing deposits to later fail.

```
89  function canUserDeposit(address _vault, address _user, uint256 _additionalAssets)
90      external
91      view
92      returns (bool)
93  {
94      if (userDepositCapByVault[_vault] == 0) {
95          return true;
96      }
97      _checkDepositLimit(IERC4626(_vault), _user, _additionalAssets);
98      return true;
99  }
100
101 function _checkDepositLimit(IERC4626 _vault, address _receiver, uint256 _additionalAssets) int
102     uint256 depositCap_ = userDepositCapByVault[address(_vault)];
103     if (depositCap_ == 0) {
104         revert VaultNotConfigured(address(_vault));
105     }
106     uint256 userShares = _vault.balanceOf(_receiver);
107     uint256 currentAssetValue = _vault.convertToAssets(userShares);
108     uint256 newTotal = currentAssetValue + _additionalAssets;
109     if (newTotal > depositCap_) {
110         revert UserDepositLimitExceeded(_receiver, newTotal, depositCap_);
111     }
112 }
```

As a result, a user calling `canUserDeposit()` may receive a positive response (indicating the deposit is valid), but when proceeding with the actual deposit, the transaction will unexpectedly revert. This inconsistency can break integrator flows, front-end UX, or off-chain validation logic that depends on `canUserDeposit()` to pre-check user eligibility before performing a deposit.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Ensure consistent logic across `canUserDeposit()` and `_checkDepositLimit()` when the deposit cap is zero.

Remediation Comment

SOLVED: The **Blueprint team** resolved the issue by allowing the _depositcap to be set as zero and removing zero deposit cap check.

Remediation Hash

https://github.com/Blueprint-Finance/earn-v2-core/commit/019daafce7432439903a8ef9de6791a5459c58ae

7.4 USE OF OWNABLE INSTEAD OF OWNABLE2STEP MAY LEAD TO OWNERSHIP TRANSFER RISKS

// INFORMATIONAL

Description

The `UserDepositCapHook` contract inherits from OpenZeppelin's `Ownable`, which implements a single-step ownership transfer mechanism via the `transferOwnership()` function. This design immediately updates the owner without requiring confirmation from the new owner's address.
If the new owner address is misconfigured (e.g., due to a typo, incorrect input, or compromised address), control of the contract could be permanently lost. Additionally, single-step ownership transfers create a potential attack vector where an unintended address could become the owner if an administrative mistake occurs.

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Replace `Ownable` with OpenZeppelin's `Ownable2Step` to implement a **two-step ownership transfer pattern**.

Remediation Comment

ACKNOWLEDGED: The **Blueprint team** acknowledged this finding, also mentioning that:

> We are using multisig, so the risk of transferownership is low. Moreover this contract can be removed/replaced by vault admin in case of emergency and this contract doesn't hold any funds. It can only cause DOS in the worst case scenario. We would like skip this finding.

7.5 UNUSED SENDER AND TOTALASSETS PARAMETER IN PREMINT() FUNCTION

// INFORMATIONAL

Description

Within the `preMint()` function of the `UserDepositCapHook` contract, the parameters `sender` and `totalAssets` are declared but never used inside the function body. These parameters appear to be part of the standardized `IHook` interface definition; however, since they are not required for the current validation logic, their presence introduces unnecessary compiler warnings and slightly reduces code clarity. While this does not impact contract functionality or security, it is considered a minor code quality issue.

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

If the `sender` and `totalAssets` parameters are of no use, we can safely remove them from both the implementation and the interface to simplify the code and improve clarity.

Remediation Comment

ACKNOWLEDGED: The **Blueprint team** acknowledged this finding, also mentioning that:

> Hook interface designed consifering future requirements. so we are fine with some unused params.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.